

Computergrafik

Vorlesung 2

Hochschule Zittau/Görlitz

Christopher-Manuel Hilgner



Agenda

- Wiederholung Grafikpipeline
- Polygon Meshes
- Mesh Representation
- Subdivision
- Optimierung
- Level of Detail
- Grafikprimitive

Grafikpipeline

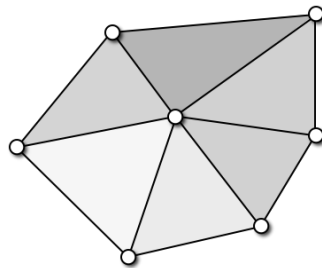
Grafikpipeline



Resources



Texturen

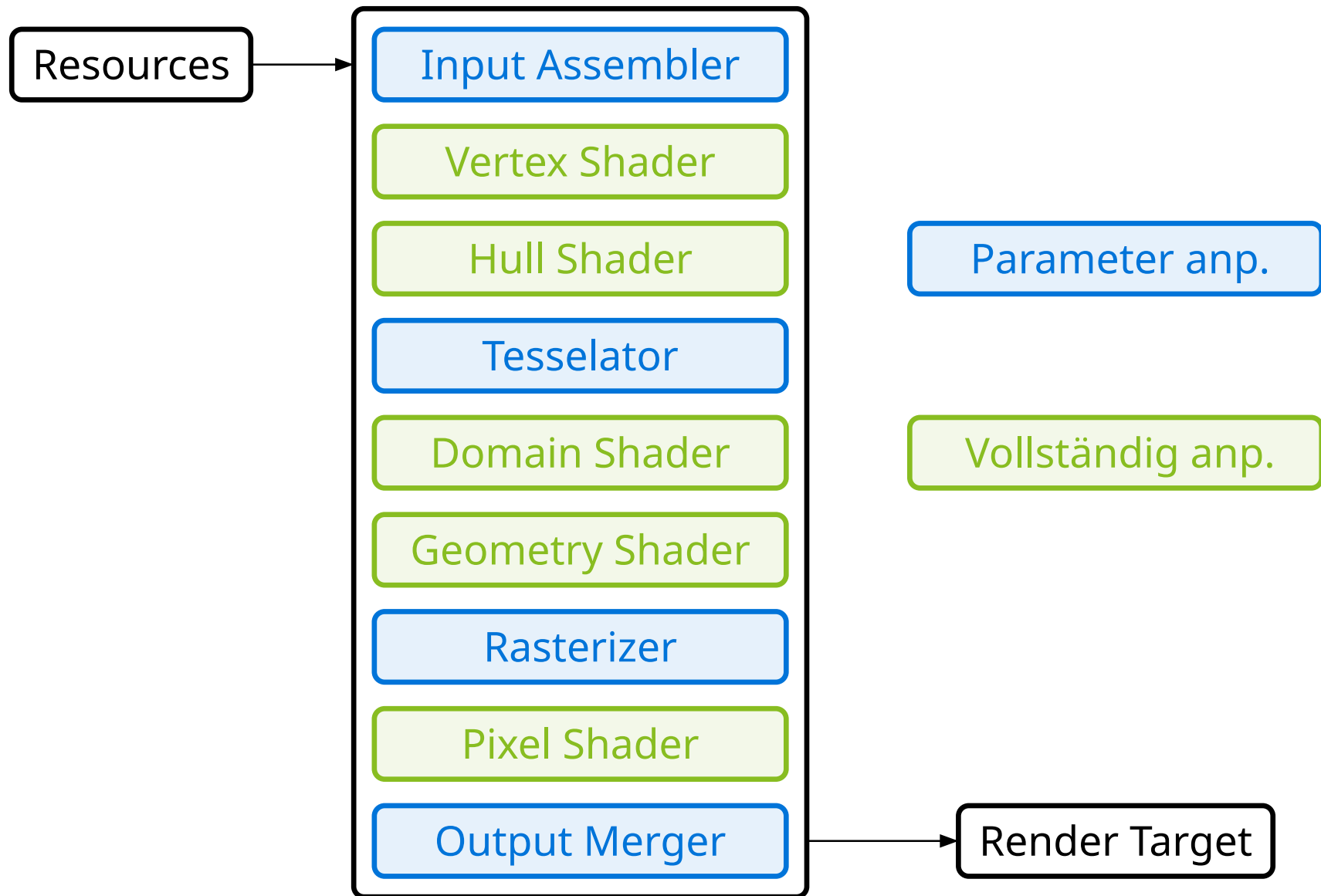


Triangle Mesh

Render Target



Grafikpipeline



Grafikpipeline

Input Assembler

→ Vertex Shader

- Input ist ein Vertex Buffer der alle Vertex Attribute aller Dreiecke eines 3D Modells enthält
- Form der Attribute muss festgelegt werden

Beispiel

Attribut	Form	Datentyp
Position	3D-Vektor	FLOAT 32
UV	2D-Vektor	FLOAT 32
Normale	3D-Vektor	FLOAT 32
ID	1	UINT 16

Grafikpipeline

Input Assembler

→ Vertex Shader

Vertex Buffer

4.55, 4.99, 5.03, 0.09, 0.56, 0.50, 0.73, 0.21, 0,
4.55, 4.99, 5.03, 0.09, 0.56, 0.50, 0.73, 0.21, 1,
4.55, 4.99, 5.03, 0.09, 0.56, 0.50, 0.73, 0.21, 2,
4.55, 4.99, 5.03, 0.09, 0.56, 0.50, 0.73, 0.21, 3,



(4.55, 4.99, 5.03), (0.09, 0.56), (0.50, 0.73, 0.21), 0,
(4.55, 4.99, 5.03), (0.09, 0.56), (0.50, 0.73, 0.21), 1,
(4.55, 4.99, 5.03), (0.09, 0.56), (0.50, 0.73, 0.21), 2,
(4.55, 4.99, 5.03), (0.09, 0.56), (0.50, 0.73, 0.21), 3,

Position

UV

Normale

ID

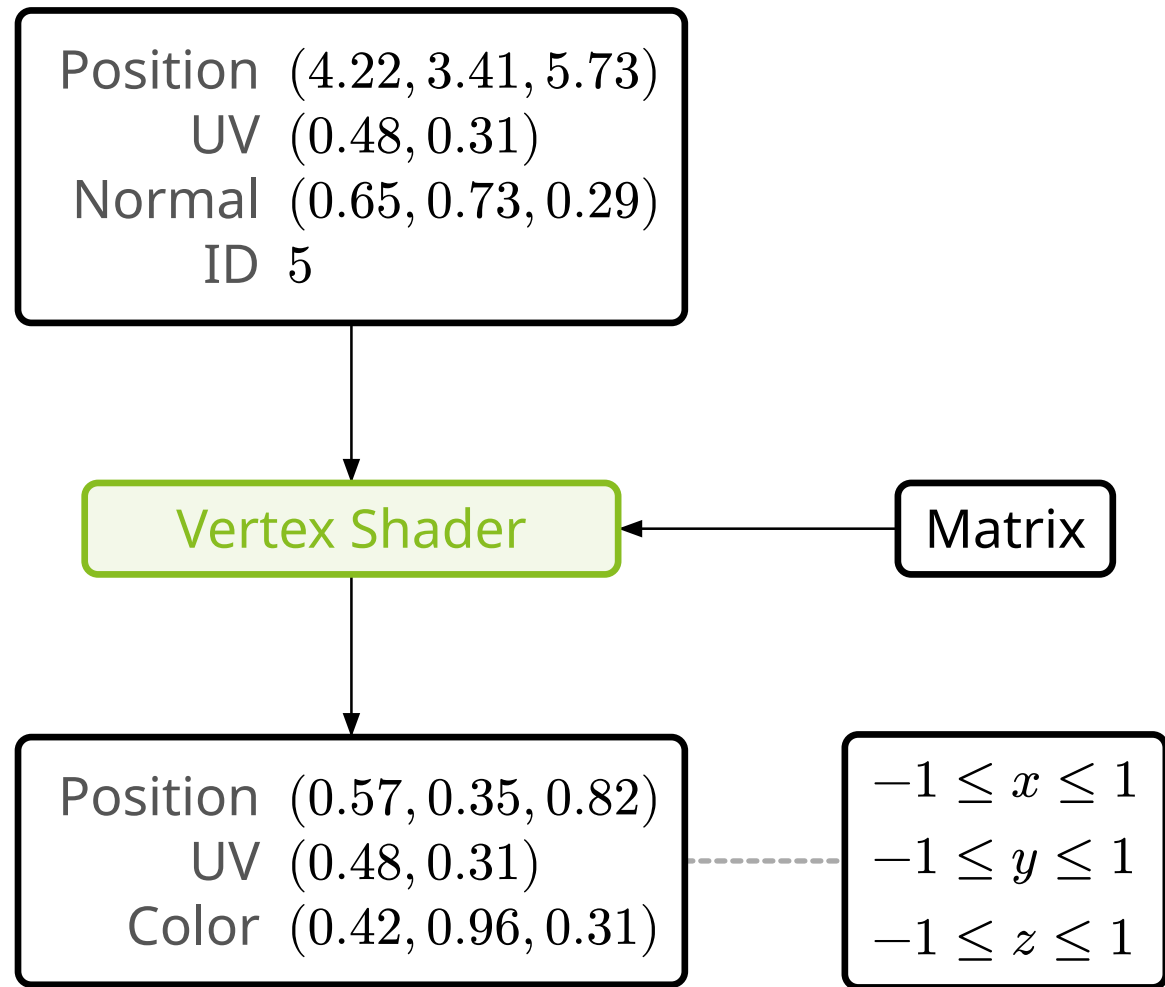
Grafikpipeline

Input Assembler ←

Vertex Shader

→ Tesselator

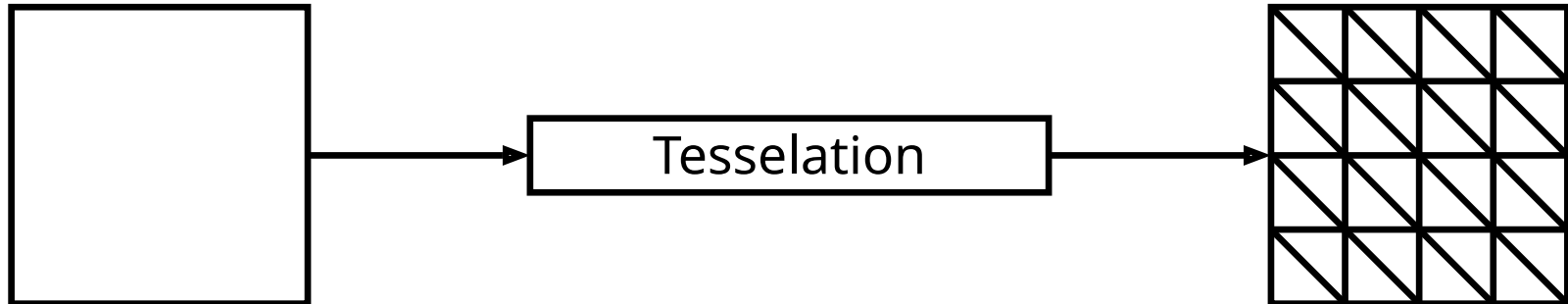
- Transformiert einen Input zu einem neuen Output
- Kann neue Daten einfügen wie z.B. *Color*
- Jede Vertex sollte nach der Transformation innerhalb von -1 und 1 liegen



Grafikpipeline

Vertex Shader ← **Tessellation** → Geometry Shader

- Umfasst: Hull Shader, Tesselator und Domain Shader
- Vereinfacht: Es nimmt eine primitive Form an und gibt eine höher aufgelöste Version davon aus
- **Hull Shader:** definiert das Muster der Oberfläche
- **Tesselator:** erstellt Triangles basierend auf dem Muster
- **Domain Shader:** positioniert die Triangles



Grafikpipeline

Tessellation ←

Geometry Shader

→ Rasterizer

- Nimmt ein Grafikprimitiv an und modifiziert es nach Spezifikation
- Primitiv kann ohne Änderung weiter gegeben werden oder mit neuen Vertices modifiziert werden
- **Beispiel 1:** Aus einem Triangle kann mit einer neuen Vertex eine Pyramide gemacht werden
- **Beispiel 2:** Nimmt eine Linie entgegen und modifiziert sie zu einer Plane aus 2 Triangles (nützlich für Haare)
- **Beispiel 3:** Nimmt einen Punkt entgegen und modifiziert ihn zu einem Quad aus 2 Triangles (nützlich für Partikelsysteme)

Grafikpipeline

Geometry Shader ←

Rasterizer

→ Pixel Shader

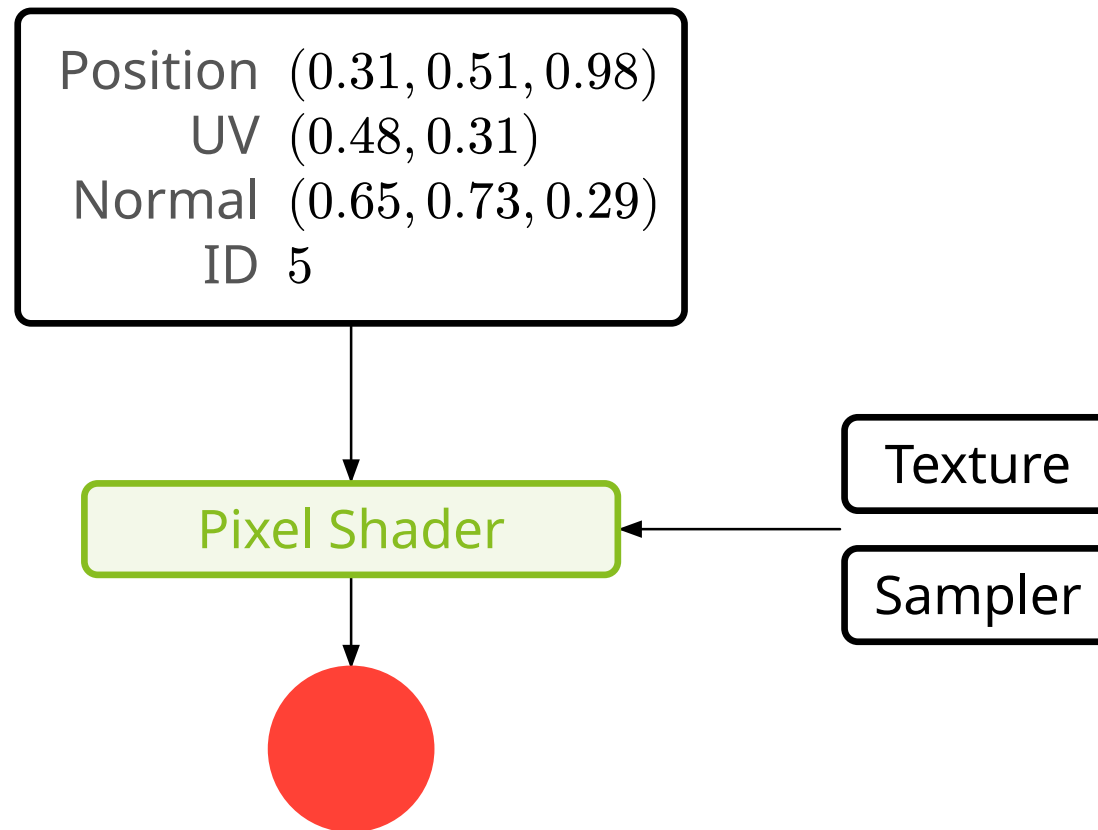
- **Input:** Jedes Triangle der Szene
- **Output:** Ein Grid an Pixeln
- Konvertierung aller Meshes in ein Pixelgrid basierend auf Kamera in der Szene
- Nutzung eines Depth Buffers um Triangles in korrekter Anordnung zu rendern
- Depth Buffer: Umwandlung der z Koordinate aller eines Dreiecks in eine Farbe (Grayscale). Je näher das Triangle ist, desto dunkler wird die Farbe.
- Rasterizer erstellt Pixelinformationen und gibt sie an den Pixel Shader weiter

Grafikpipeline

Rasterizer ←

Pixel Shader

→ Output Merger



Grafikpipeline

Pixel Shader ← **Output Merger** → Render Output

- Nimmt die Farb , Tiefen Informationen, Render Target und Depth Buffer eines Pixels
- Typischerweise ist der Output die gleiche Farbe wie der Input
- Blending von Input Farbe und Render Target Farbe kann hier geschehen

$P_{\text{depth}} < \text{Depth Buffer}$:

Output Merger schreibt Werte in Render Target und Depth Buffer.

$P_{\text{depth}} > \text{Depth Buffer}$:

Render Target Farbe wird nicht überschrieben. Die Farbe bleibt gleich.

Polygon Meshes

Polygon Meshes

Definitionen



Definition

Polygon

Eine vielseitige, ebene Figur, die aus Vertices und Edges besteht. Die Edges sind dabei in einer geschlossenen Kette miteinander verbunden.

Polygon Meshes

Definitionen



Definition

Vertices

Darstellung eines Punktes im 2D oder 3D Raum. An einer Vertex kreuzen bzw. treffen sich zwei oder mehr Edges.

$A(x, y)$ in 2D, $A(x, y, z)$ in 3D

Polygon Meshes

Definitionen



Definition

Edges

Liniensegmente im 2D oder 3D Raum, die zwei Vertices, A und B miteinander verbinden.

$$(A_x, A_y) \text{ und } (B_x, B_y)$$

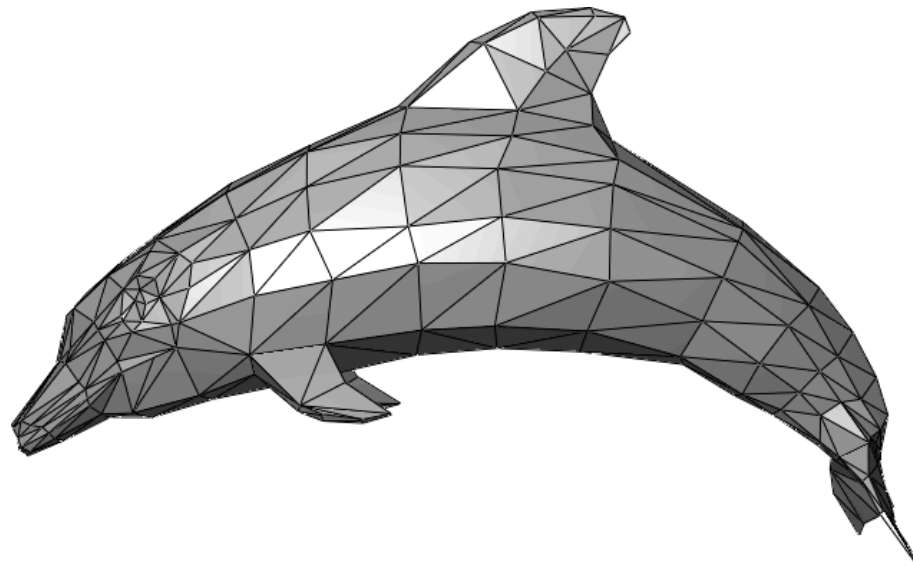
Polygon Meshes

Klassisch und allgegenwärtig in Computergrafik

- Erstellen polygonaler Objekte ist unkompliziert
- Es gibt visuell effiziente Algorithmen zur Erzeugung schattierter Bilder,
- Kann auf Basis anderer Daten erstellt werden

Schwierigkeiten bei Polygon Meshes:

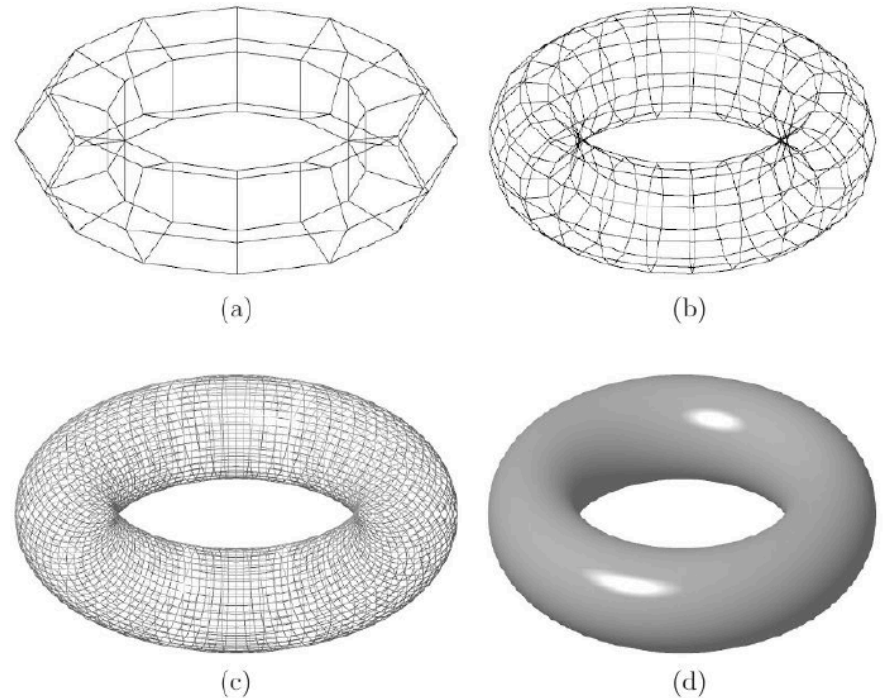
- geometrische Genauigkeit
- Formmanipulation



Polygon Meshes

Tori in verschiedenen
Polygonauflösungsstufen:

- (a) 50 Quads
- (b) 200 Quads
- (c) 3200 Quads
- (d) 819200 Quads (da hier das Drahtgitter so fein ist, dass auf jedes Pixel mindestens eine Gitterlinie fällt, sieht es so aus, als ob der Torus eine geschlossene Oberfläche hätte)



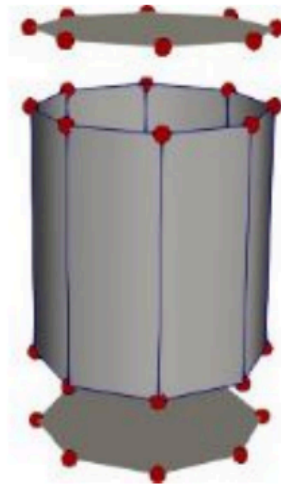
Polygon Meshes 🚫



Objekt



Surface



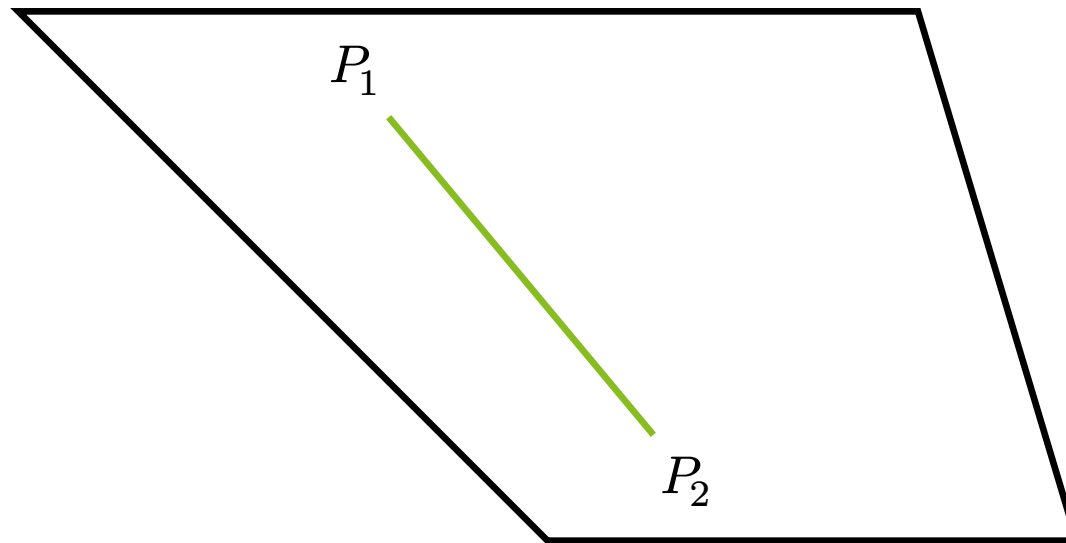
Polygone



Edges/
Vertices

Konvexe Polygone

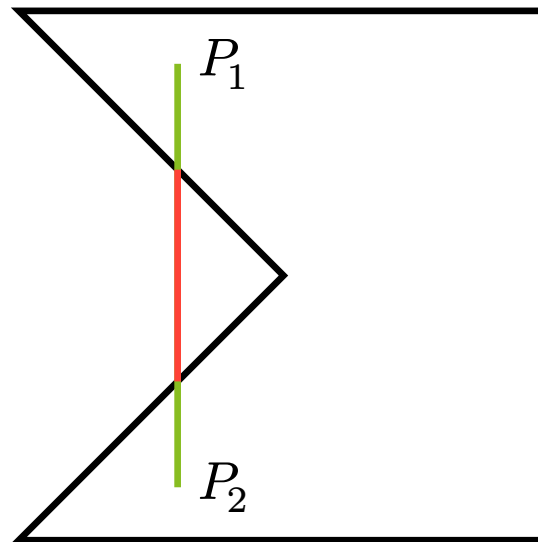
- Bei zwei beliebigen Punkten P_1 , P_2 innerhalb des Polygons befinden sich alle Punkte, die auf dem Liniensegment zwischen P_1 und P_2 liegen, innerhalb des Polygons.



Konkave Polygone

- Konkaves Polygon - Ein Polygon, das nicht konvex ist.

Well no shit, captain obvious!

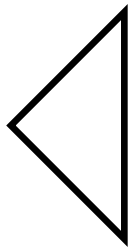


(Nicht) Einfache Polygone

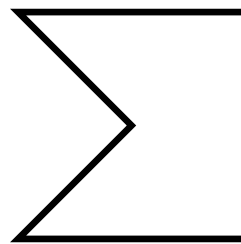
- **Einfache Polygone** - Kanten kreuzen sich nicht
- **Nicht einfache Polygone** - Kanten kreuzen sich
 - Zwei verschiedene OpenGL-Implementierungen können nicht einfache Polygone unterschiedlich darstellen. OpenGL prüft nicht, ob Polygone einfach sind.

Deshalb oft Triangles

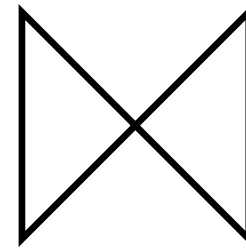
- Triangle-Polygon garantiert **konvex** und **einfach** (und planar in 3D)



Triangle



Einfaches (konkaves)
Polygon



Nicht einfaches
Polygon

Mesh Representation

Mesh Representation

- Meshes können unterschiedlich abgespeichert und repräsentiert werden

Mesh Representations:

- Independent Faces
- Vertex and Face Table
- Adjacency Lists
- Winged Edge

Mesh Representation

Independent Faces

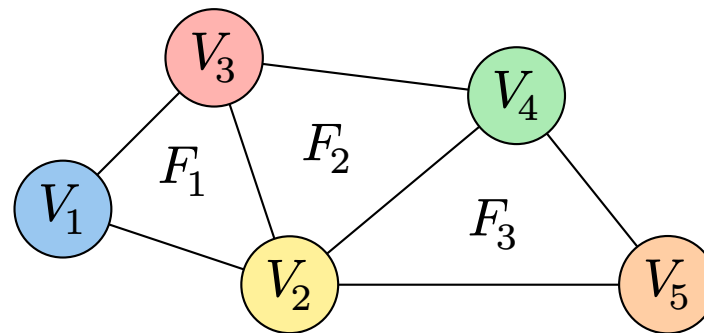
- Jede Fläche listet Vertex-Koordinaten auf
 - Redundante Eckpunkte
 - Keine Topologie-Informationen

Vertex und Face-Table

- Jede Fläche listet Vertex-Referenzen auf
 - Gemeinsam genutzte Eckpunkte, kompaktere Darstellung
 - Immer noch keine Topologie-Informationen

Mesh Representation

Vertex und Face-Table



Vertex Table	
v1	x1 y1 z1
v2	x2 y2 z2
v3	x3 y3 z3
v4	x4 y4 z4
v5	x5 y5 z5

Face Table	
F1	v1 v2 v3
F2	v2 v4 v3
F3	v2 v5 v4

Mesh Representation

Adjacency Lists

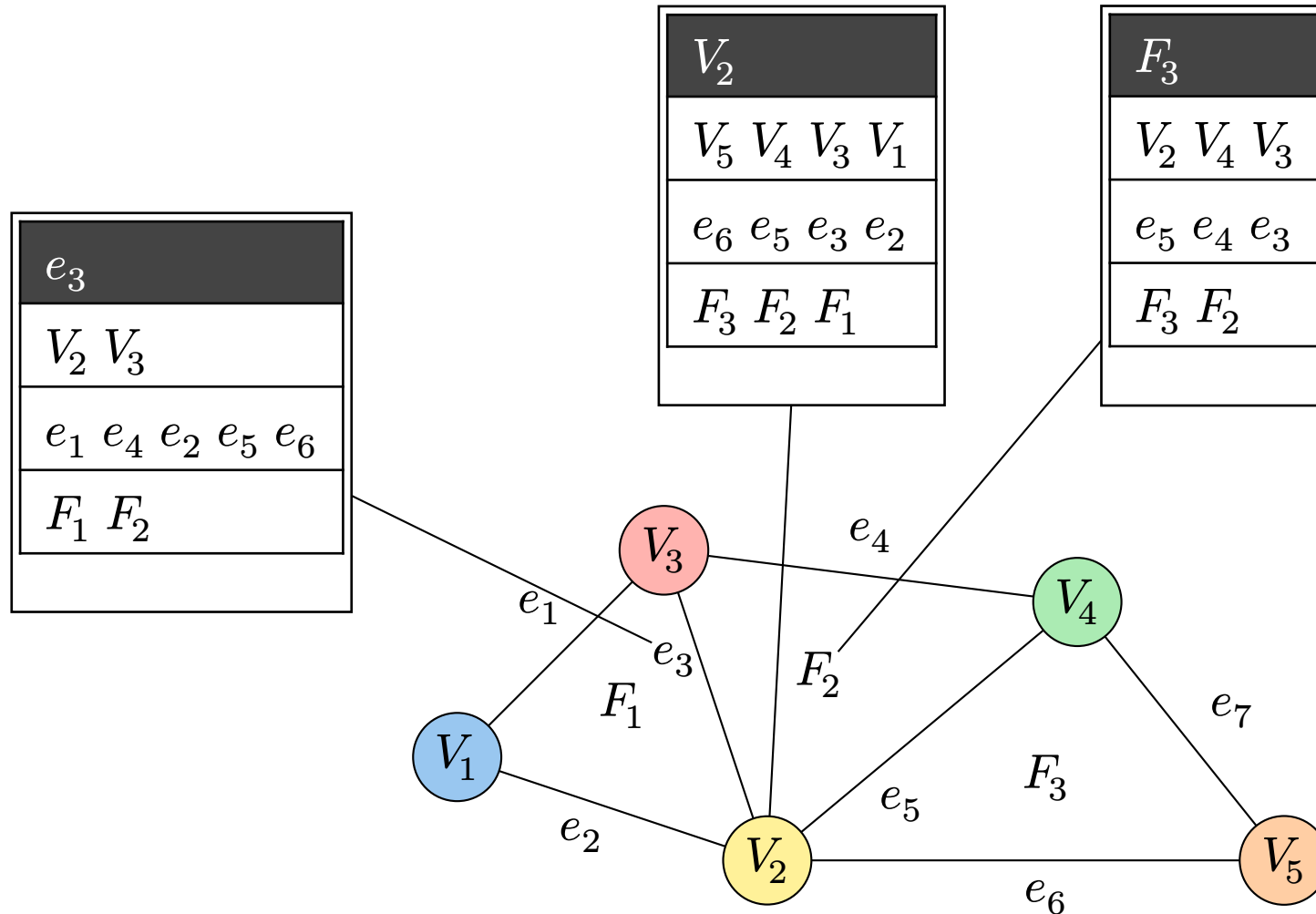
- Alle Vertex-, Kanten- und Flächenadjazenzen speichern
- Effizientes Traversieren der Topologie
- Zusätzlicher Speicher benötigt

Winged Edge

- Erweiterte Mesh-Darstellung
- Speichert nur einige Adjazenzbeziehungen; kann auf Anfrage andere ableiten
- Wenig zusätzlicher Speicherplatz (feste Datensätze)

Mesh Representation

Adjacency Lists

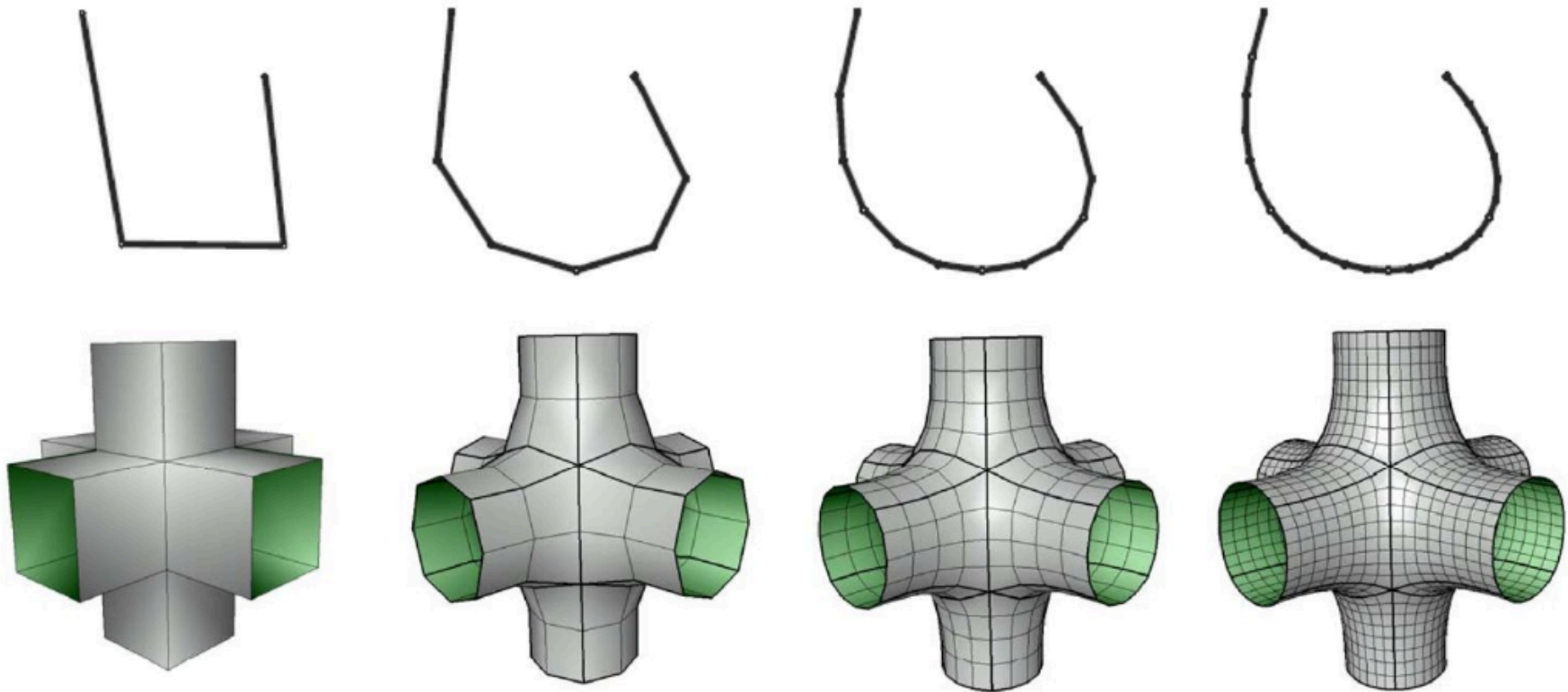


Subdivision

Subdivision

Frage: Wie macht man eine glatte Kurve?

Antwort: Schneiden der Ecken der Polyline so lange, bis Sie etwas erhalten, das keine Ecken mehr hat also glatt bzw. smooth ist.



Subdivision

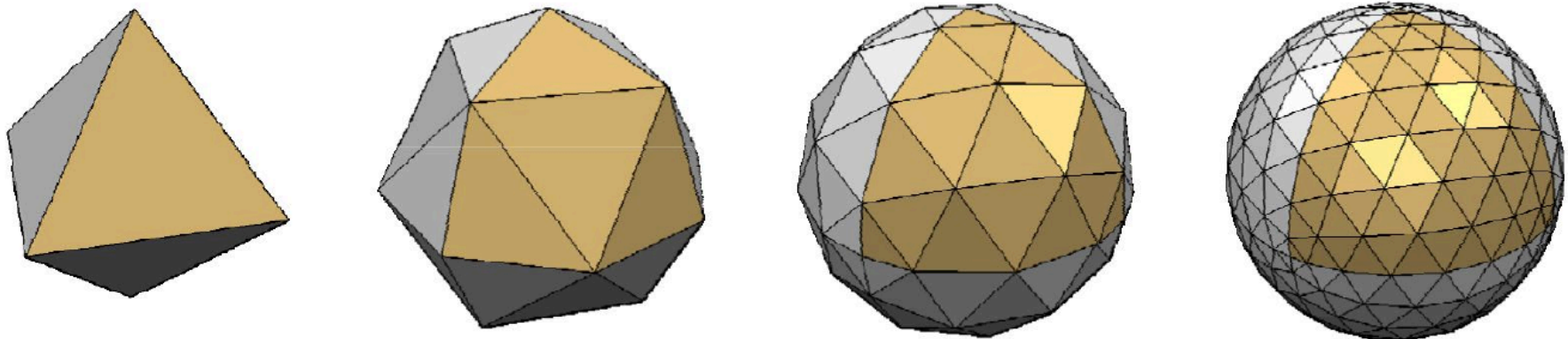
Eigenschaften:

- Genau
- Lokale Unterstützung von Problempunkten
- Affine Invariante
- Arbiträre Topologie
- Garantierte Kontinuität
- Effiziente Anzeige

Subdivision

Grobmaschiges Mesh & Unterteilungsregel

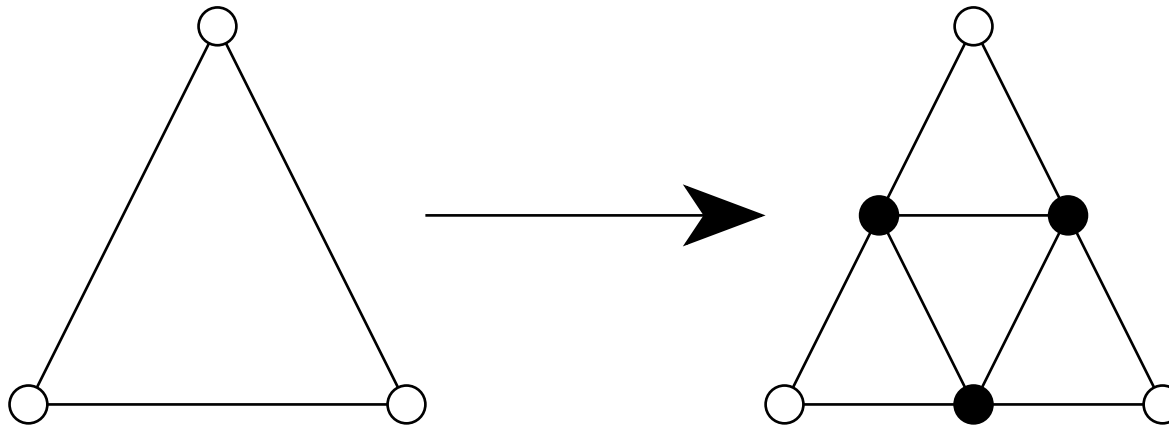
- Glatte Oberfläche als Grenze der Folge von Verfeinerungen definieren



Subdivision

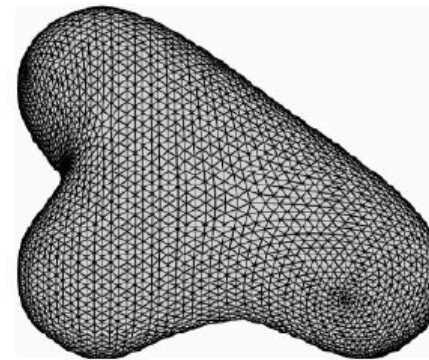
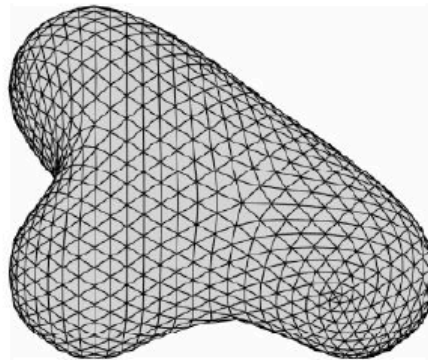
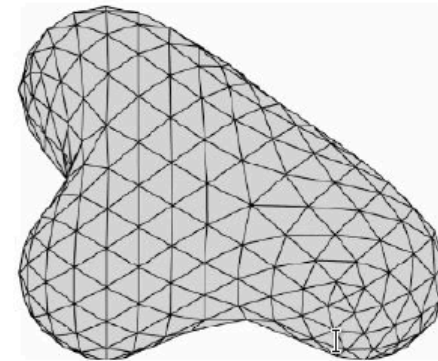
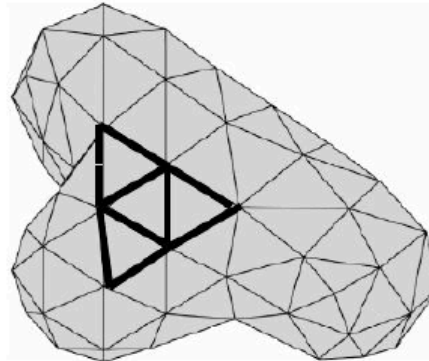
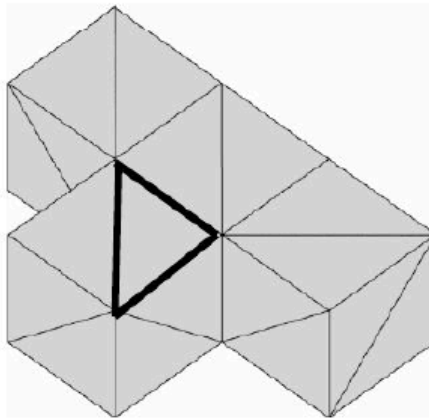
Loop Unterteilungsregel

Jedes Dreieck in 4 Dreiecke verfeinern, indem jede Kante geteilt und neue Eckpunkte verbunden werden.



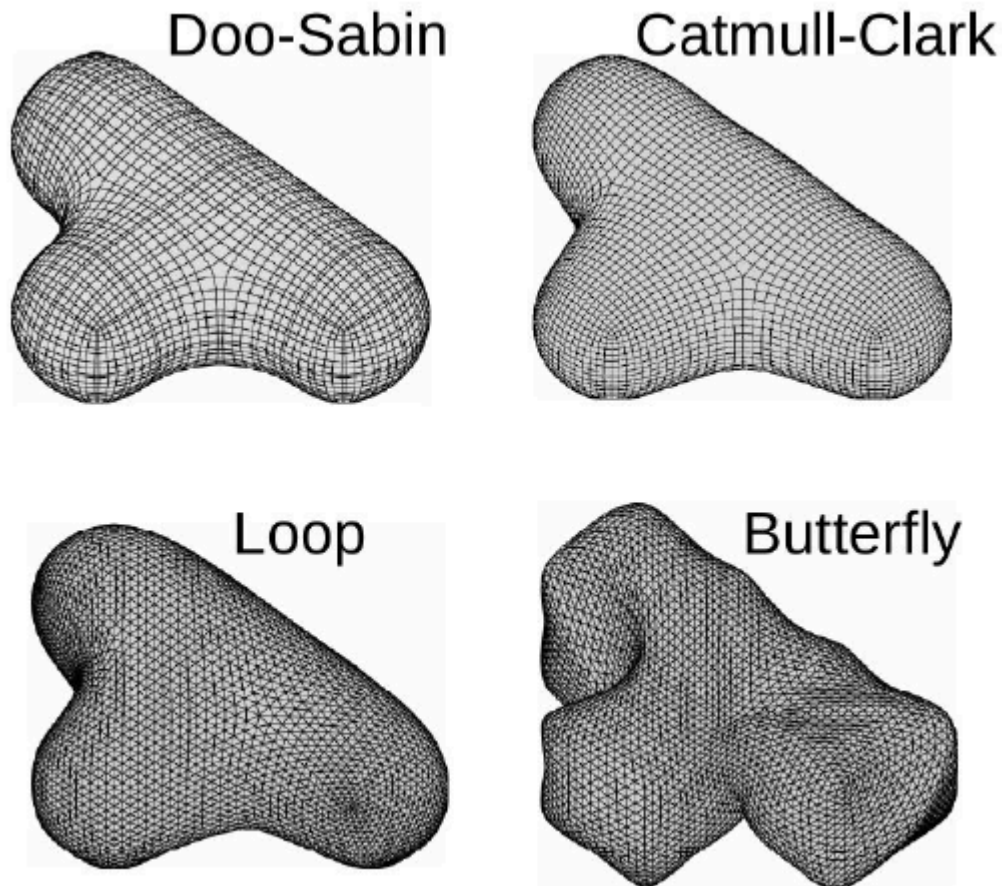
Subdivision

Loop Unterteilungsregel



Subdivision

- Viele weitere Methoden möglich („Subdivision Zoo“)
- Blender verwendet vor allem den Catmull-Clark Algorithmus



Subdivision

Catmull-Clark

Catmull-Clark

Übersicht

- Rekursiver Ansatz zur Verfeinerung von Meshes
- Die fertige Mesh wird nur aus Vierecken bestehen und weicher aussehen als ursprünglich
- Je mehr Durchgänge, desto abgerundeter wird die Mesh

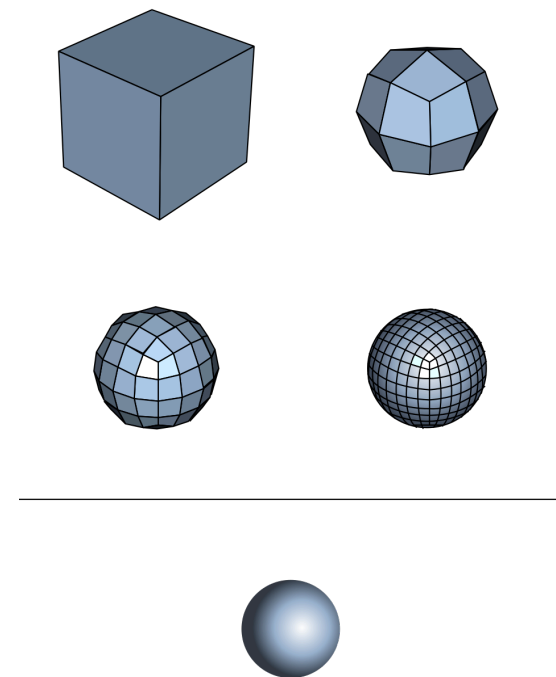


Figure 21: 3-stufige Subdivision an einem Würfel. Mit jedem Schritt nähert sich der Würfel der Form einer Kugel an.

Catmull-Clark

Abfolge

Für jedes Face wird ein *Face Point* hinzugefügt.

Jeder *Face Point* ist der Durchschnitt aller Punkte des jeweiligen Faces

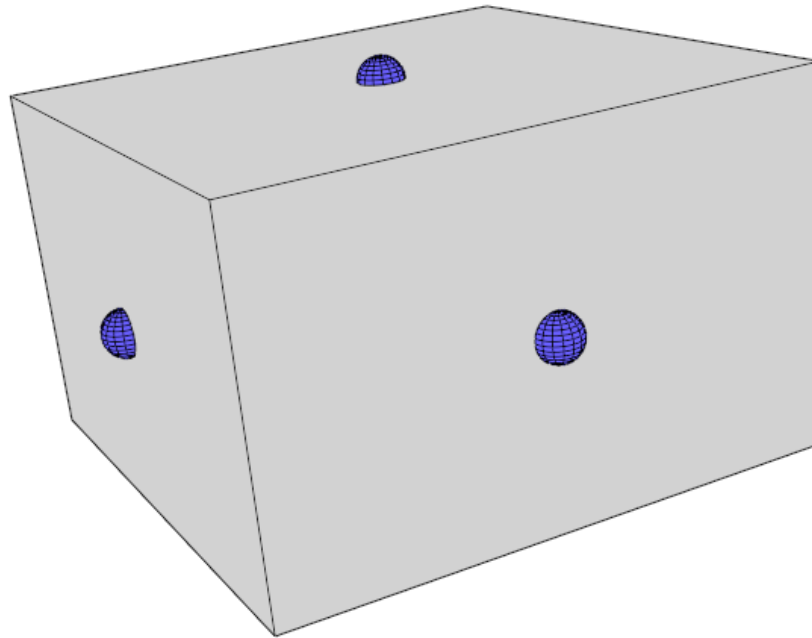


Figure 22: Blaue Spheres als *Face Points*

Catmull-Clark

Abfolge

Für jede Edge wird ein *Edge Point* hinzugefügt.

Jeder *Edge Point* ist der Durchschnitt der zwei benachbarten *Face Points* (A, F) und die beiden Endpunkte der Kante (M, E) .

$$\frac{A + F + M + E}{4}$$

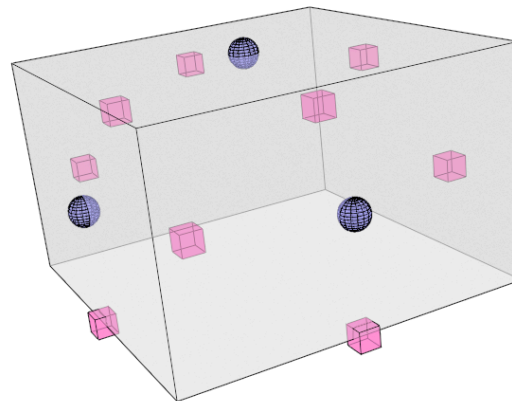


Figure 23: Magenta Würfel als *Edge Points*

Catmull-Clark

Abfolge - *Vertex Points*

Erstelle neue *Vertex Points*. Für jeden ursprünglichen Punkt P , nehme den Durchschnitt F aller n erstellen *Face Point*, deren Faces P berühren und nehme den Durchschnitt R aller n Kanten-Mittelpunkte der ursprünglichen Kanten, die P berühren. Jeder ursprüngliche Punkt wird zum neuen *Vertex Points* V verschoben.

$$V = \frac{F + 2R + (n - 3)P}{n}$$

Catmull-Clark

Abfolge - *Vertex Points*

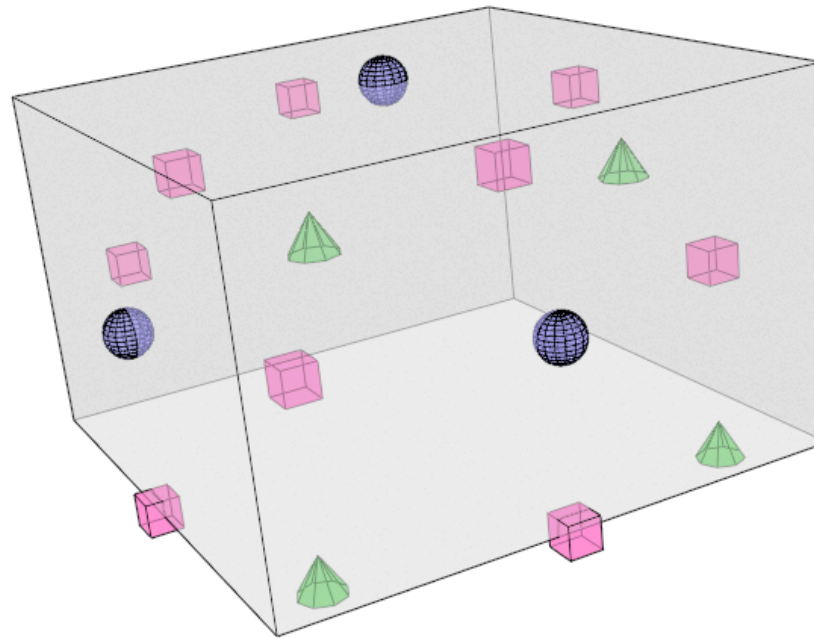


Figure 24: Grüne Kegel als neue *Vertex Points*

Catmull-Clark

Abfolge - Erstellen von neuen *Edges* und *Faces*

1. Verbinde jeden neuen *Face Point* mit den neuen *Edge Points* aller ursprünglichen Edges, die das ursprüngliche Face definieren

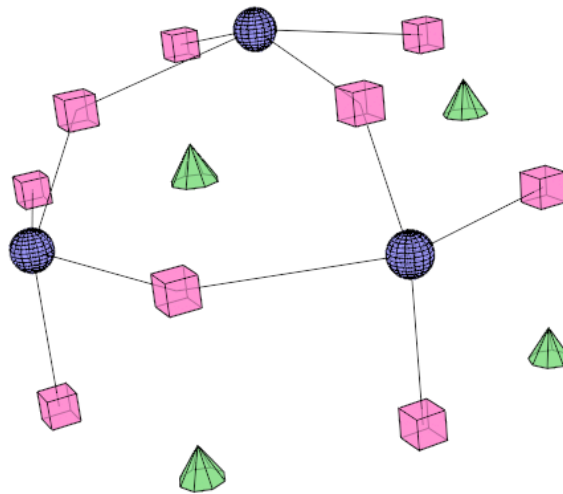


Figure 25: Neue Edges, 4 pro *Face Point*

Catmull-Clark

Abfolge - Erstellen von neuen *Edges* und *Faces*

2. Verbinde jeden neuen *Vertex Point* mit den neuen *Edge Points* aller ursprünglichen Kanten, die am ursprünglichen Knoten inzident sind

Inzident: Eine Kante e ist inzident zu einem Knoten v , wenn v einer der Endpunkte von e ist.

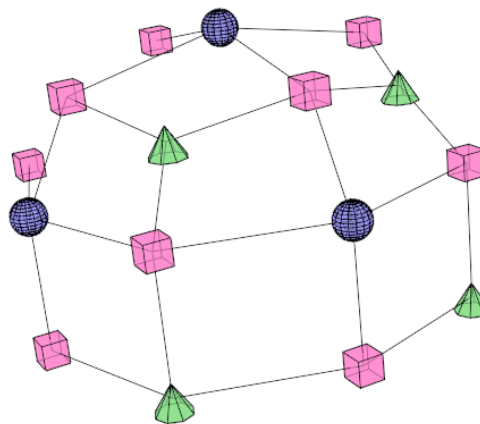


Figure 26: 3 neue Edges pro Vertex

Catmull-Clark

Abfolge - Faces erstellen

3. Definiere neue Faces, die von Edges umschlossen sind.

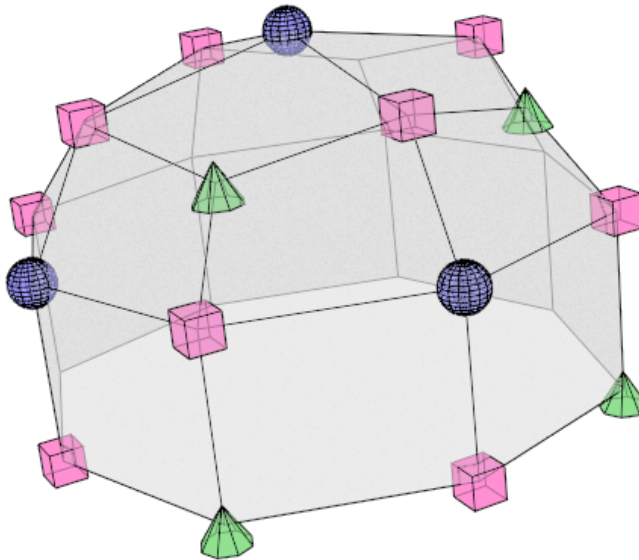


Figure 27: Finale Faces der Mesh

Subdivision

Zusammenfassung

Subdivision

Alle Unterteilungsschemata haben 2 Schritte:

1. **Aufspaltung oder Verfeinerungsschritt:** Topologische Regel, die neue Eckpunkte einführt und die Konnektivität modifiziert
2. **Schritt der Mittelwertbildung oder Glättung:** Geometrische Regel, die die Positionen für neue oder alle Scheitelpunkte durch gewichtete Mittelwerte berechnet.

Adaptive Subdivision

Adaptive Approximation: Oberfläche wird nur dort unterteilt, wo die Topologie der Oberfläche mehr Details erfordert

- Auswahl der kritischen Netzbereiche (Scheitelpunkte) von Hand
- oder automatische Erkennung von gekrümmten (weiter unterteilen) vs. flachen (nicht weiter unterteilen) Bereichen

Polygonal Meshes vs. Subdivision

- Erstellung einer Subdivision Surface durch Modifier
- Zwei Arten werden unterstützt:
 - HSDS Modifier bringt hierarchische Subdivision Surfaces
 - MeshSmooth Modifier bringt Weichzeichnung
- Modifier werden am besten als Finishing Tool genutzt

HSDS

Richtlinie 1

The HSDS modifier implements Hierarchical subdivision Surfaces. It is intended primarily as a finishing tool rather than as a modeling tool. For best results, perform most of your modeling using low-polygon methods, and then use HSDS to add detail and adaptively refine the model.

Quelle: 3DS Max Help

Optimierung

Polygon Mesh Optimierung

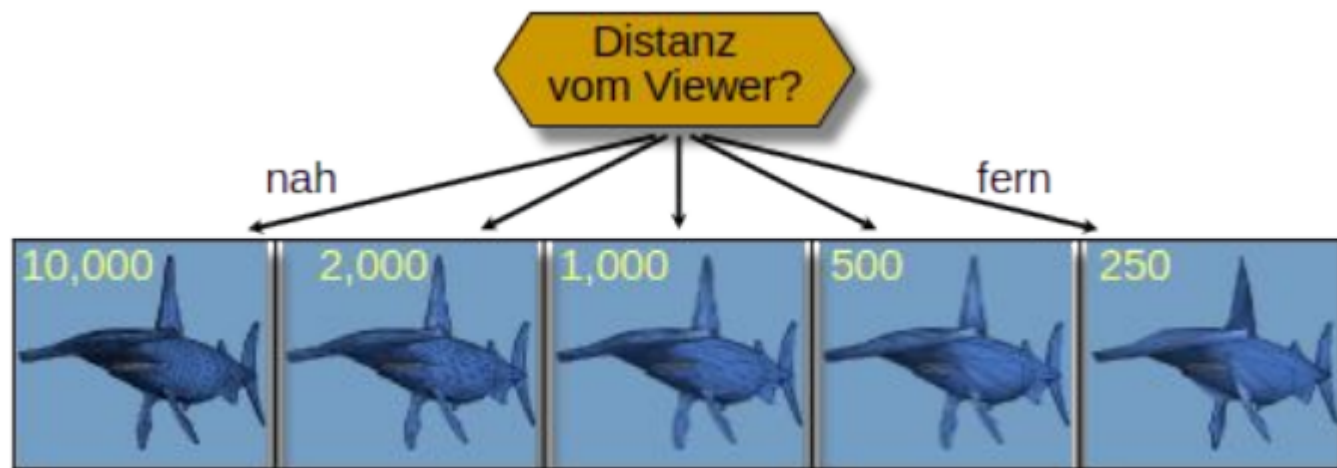
- Hauptnachteil von Polygonnetzen:
 - oft hohe Polygonanzahl zur Synthese eines Objekts für eine qualitativ hochwertige Edition
 - Heutige Modelle > 10 Mio Polygone möglich

„It makes no sense to use 500 polygons in describing an object if it covers only 20 raster units on the display ... For example, when we view the human body from a very large distance, we might need to present only specks for the eyes, or perhaps a block for the head, totally eliminating the eyes from consideration“

(James H. Clark, 1976)

Polygon Mesh Optimierung

- Polygone auf ein Niveau reduzieren, das der geforderten Qualität angemessen ist
 - Anwendung: Grad der Detailannäherung
 - [Clark76, Funkhouser93]
 - VRML, OpenSceneGraph: LOD-Knoten
- Problem: „PopUp-Effekt“

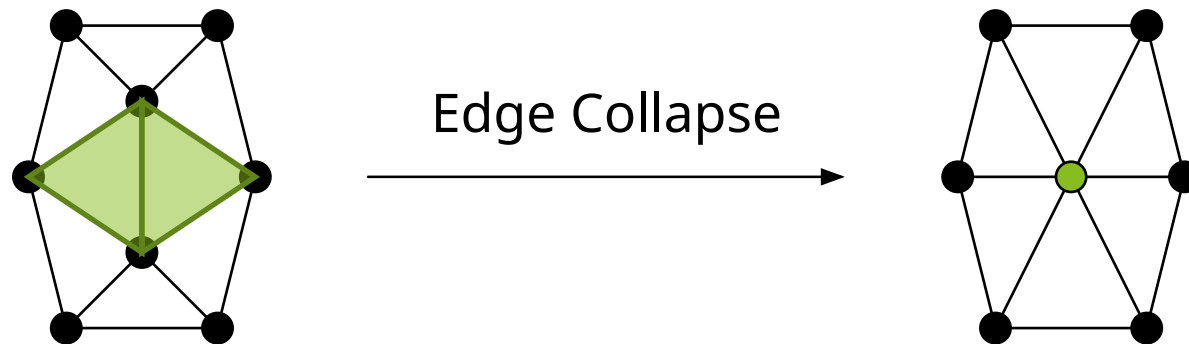


Progressive Meshes

Edge Collapse

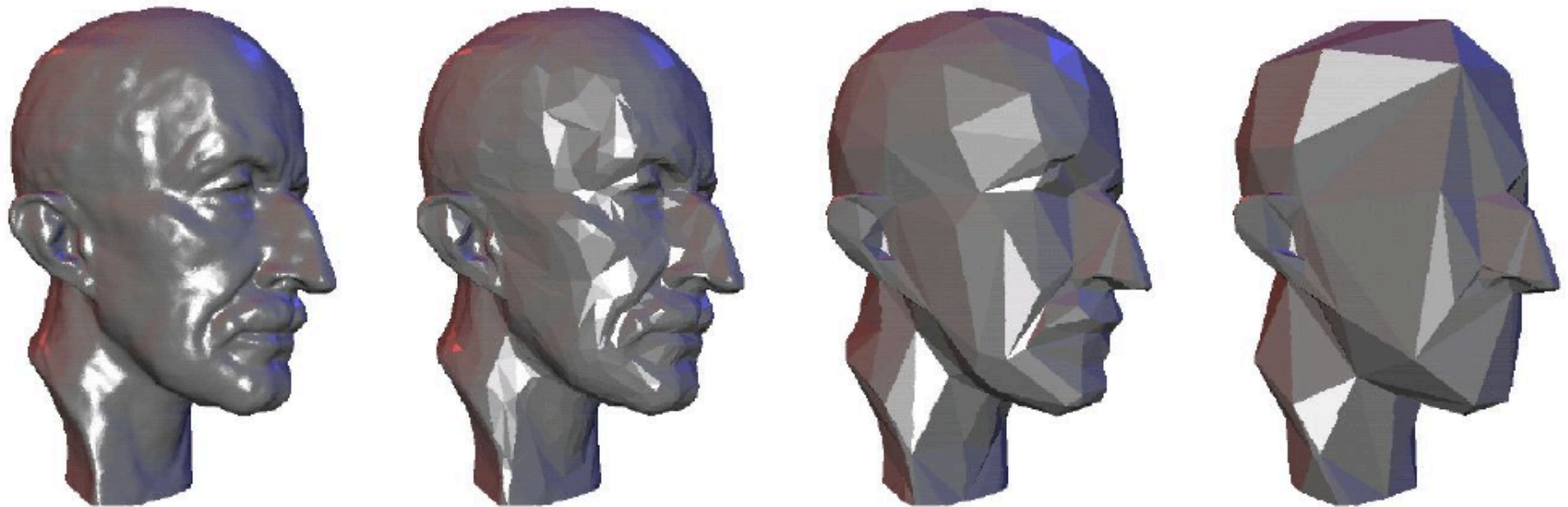
Idee: Sequenz von Kanteneinbrüchen (edge collapses) anwenden

- Erstellung einer Hierarchie von Objekten
- Blender: Decimate Modifier
- In modernen Engines (bspw. UE4) teilweise integriert



Progressive Meshes

Edge Collapse



Progressive Meshes

Edge Collapse & Vertes Split

Welche Kanten sind auszuwählen

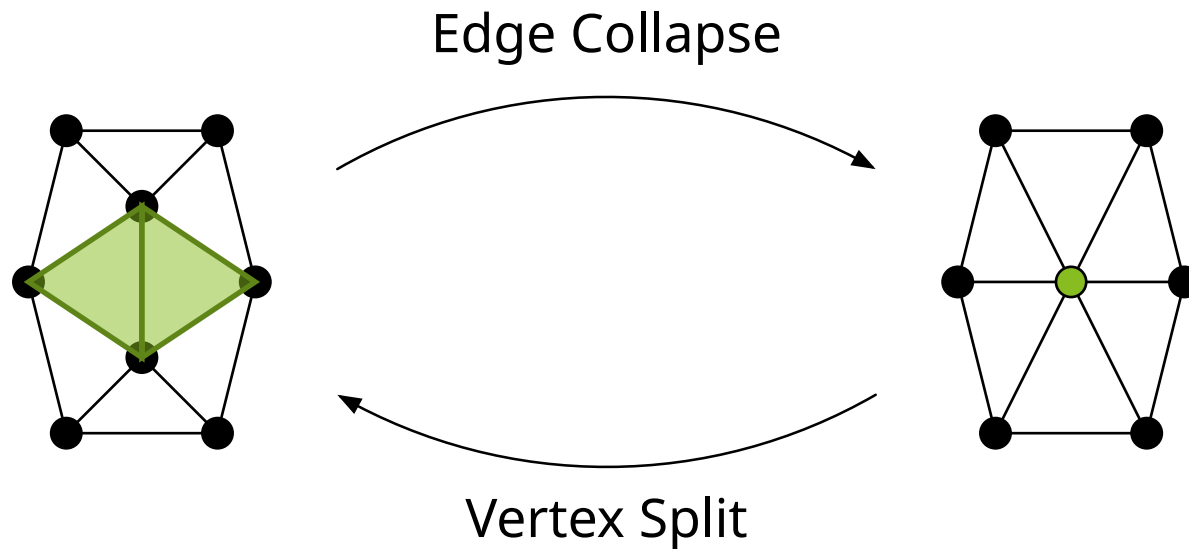
- einfache Heuristik: Kanten, die enge Polygone mit ähnlicher Orientierung verbinden
- oder komplexere Heuristiken

Kantenkollaps ist umkehrbar!

- inverse Operation: Vertex Split
- kann erforderliche Vertex-Informationen in einem gröberen Netz speichern
- verlustfrei

Progressive Meshes

Edge Collapse & Vertex Split



Progressive Meshes

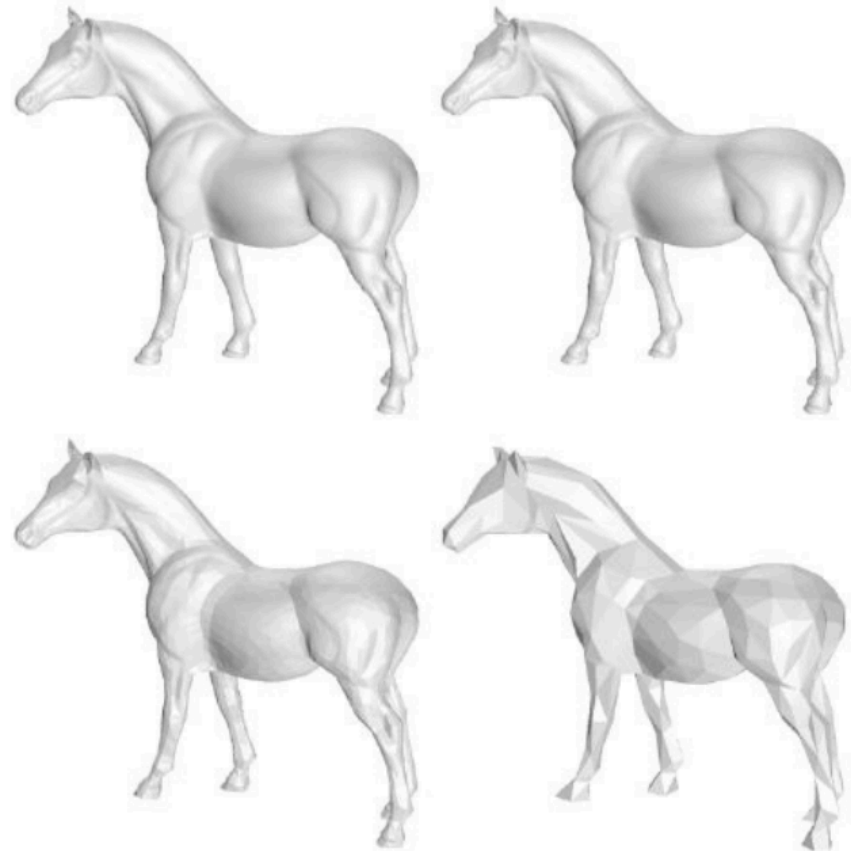
Anwendung:

- continuous level of detail
- progressive Übertragung / Transmission
- selektive Verfeinerung, z.B. betrachterabhängig
- Mesh compression

LOD

Continuous Level of Detail (CLOD)

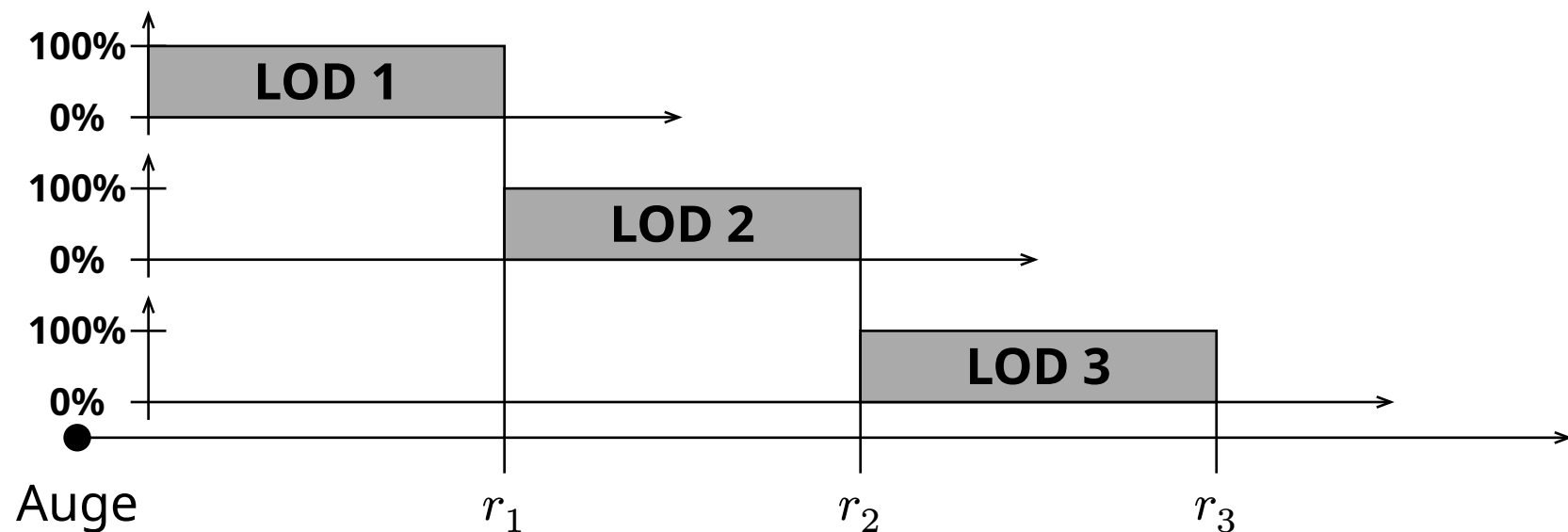
- Modell mit mehreren Auflösungen, das durch Vereinfachung des ursprünglichen Modells (in der linken oberen Ecke) erhalten wurde
- Anzahl der Dreiecke beträgt jeweils
 - 96966
 - 13334
 - 3334
 - 668



Level of Detail

Switch-LoD

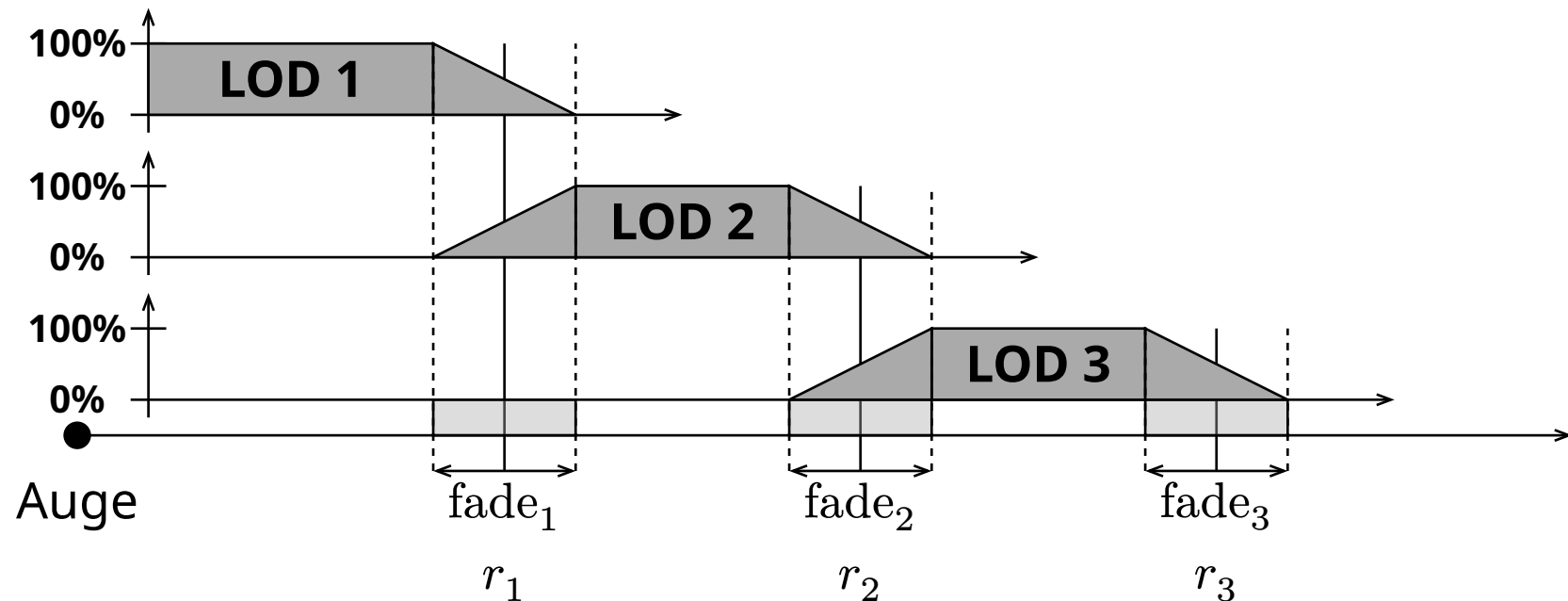
- Einfachste Form zur Anwendung von LoD-Stufen
- Harte Abstandsgrößen definieren Anzeige der Detailstufen



Level of Detail

Fade-LoD

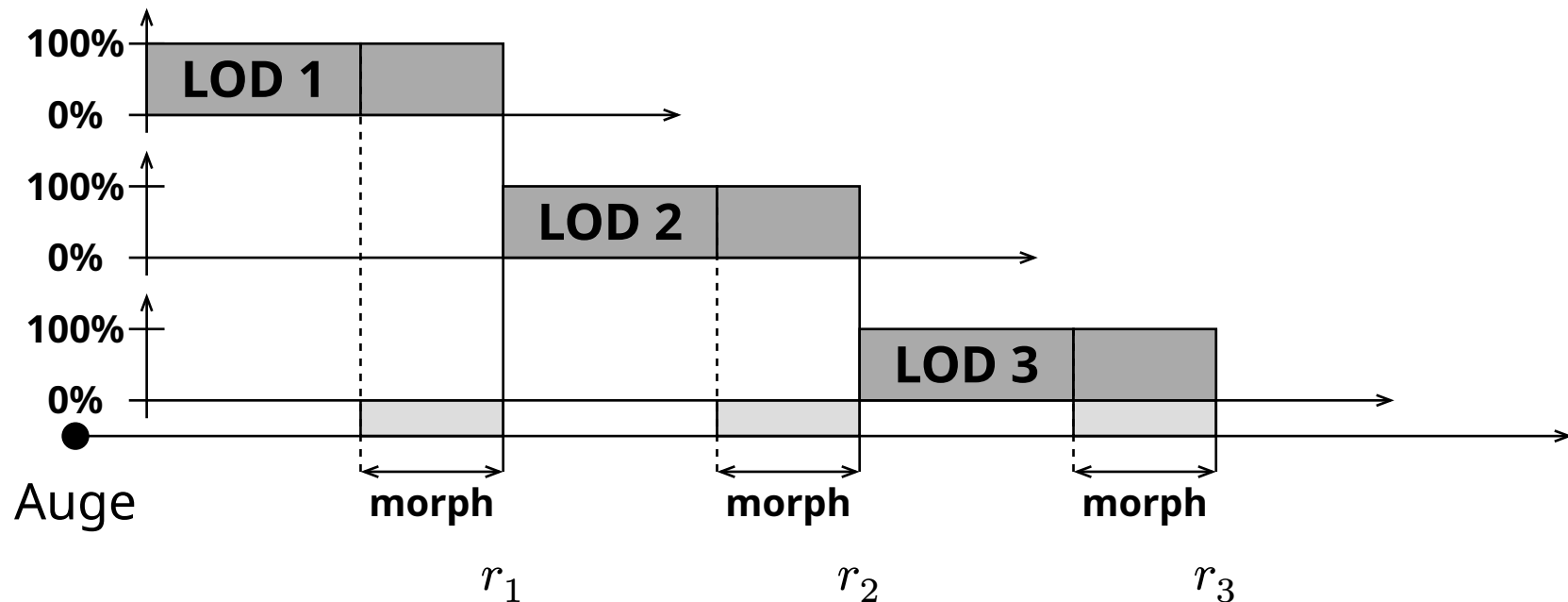
- “Überblendtechnik”, um LoD-Stufen optisch zu verschmelzen
- Verschmelzung durch Farb-Mischung / alpha blending



Level of Detail

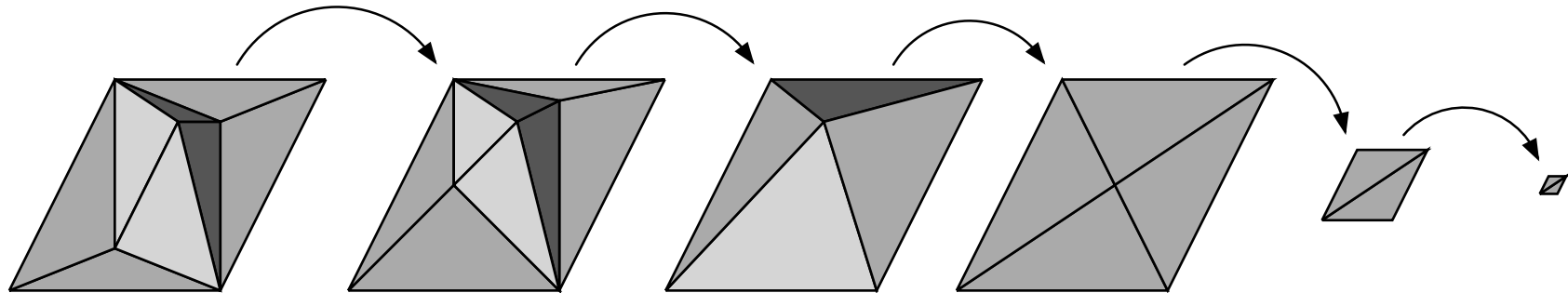
Morph-LoD

- Kontinuierliche Formveränderung zwischen LoD-Stufen
- Bei komplexen Modellen erheblicher Rechenaufwand → Vertex Shader



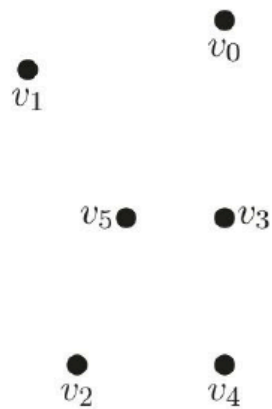
Level of Detail

Morph-LoD

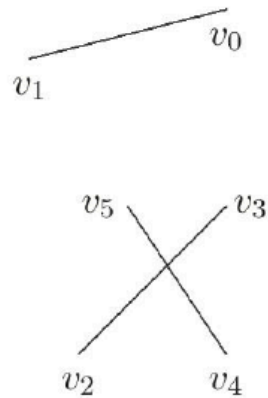


Grafikprimitive

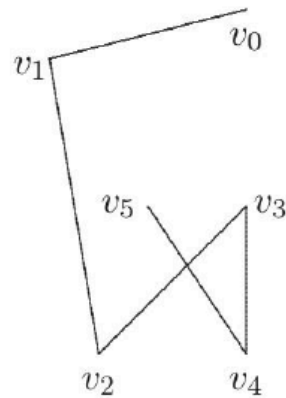
Grafikprimitive



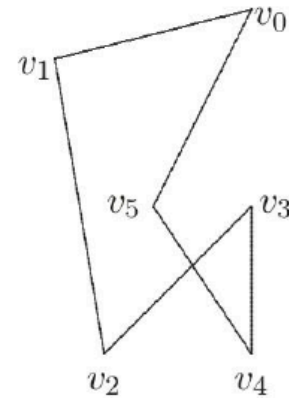
GL_POINTS



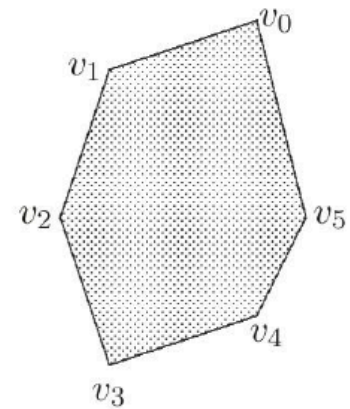
GL_LINES



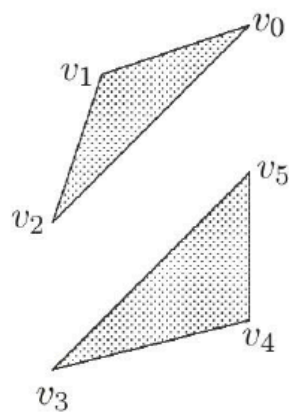
GL_LINE_STRIP



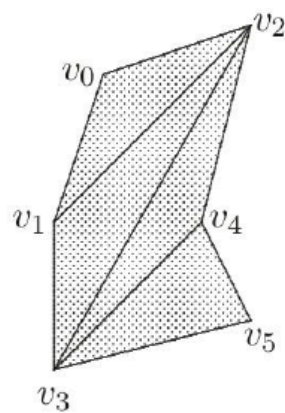
GL_LINE_LOOP



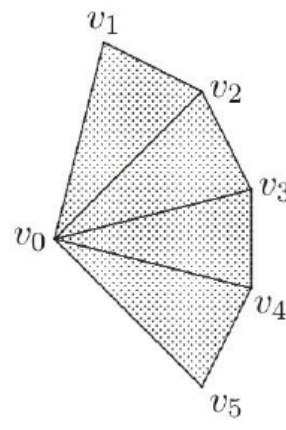
GL_POLYGON



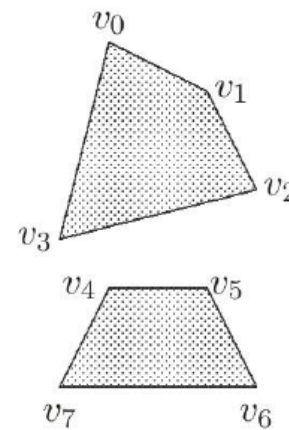
GL_TRIANGLES



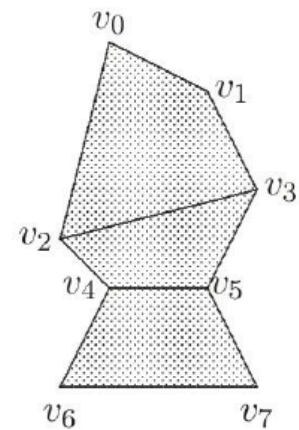
GL_TRIANGLE_STRIP



GL_TRIANGLE_FAN



GL_QUADS



GL_QUAD_STRIP

Grafikprimitive

- OpenGL Compatibility Profile (10 Primitive) vs. Core Profile (7 Primitive)
- Vulkan reduziert auf 6 Primitive, zusätzliche Primitive für spezielle Shader möglich (bspw. _LINE LIST WITH ADJACENCY)

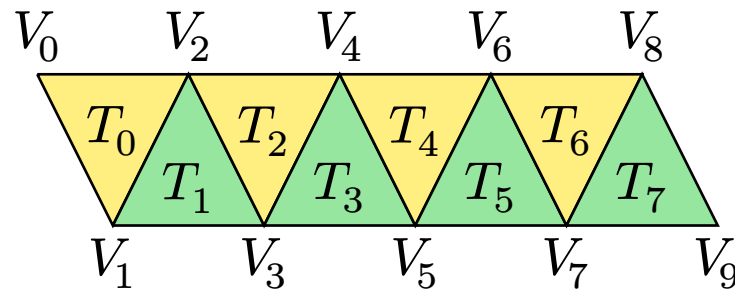
Grafik-Primitive in Vulkan	Grafik-Primitive in OpenGL
VK PRIMITIVE TOPOLOGY POINT LIST	GL POINTS
VK PRIMITIVE TOPOLOGY LINE LIST	GL LINES
VK PRIMITIVE TOPOLOGY LINE STRIP	GL LINE STRIP
VK PRIMITIVE TOPOLOGY TRIANGLE LIST	GL TRIANGLES
VK PRIMITIVE TOPOLOGY TRIANGLE STRIP	GL TRIANGLES STRIP
VK PRIMITIVE TOPOLOGY TRIANGLE FAN	GL TRIANGLES FAN

Grafikprimitive

Triangle Strips

Triangle Strips

- Ohne Strips: $8 \text{ triangles} \times 3 \text{ vertices} = 24 \text{ vertices}$
- Mit Strips: 1 vertex per triangle anstelle von 3



Was wir an die Graphic-Hardware senden:

Startkosten: V_0, V_1

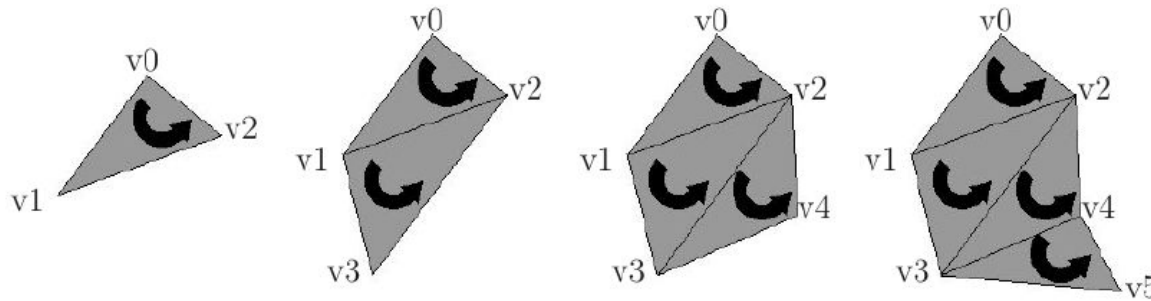
Dann: $T_0 = \{V_2\}, T_1 = \{V_3\}, T_2 = \{V_4\}, T_3 = \{V_5\}, T_4 = \{V_6\}, T_5 = \{V_7\}, T_6 = \{V_8\}, T_7 = \{V_9\},$

Triangle Strips

- 10 Vertices statt 24
 - $\frac{100 \times 10}{24} = 41.7\%$ der Daten
 - $\frac{10}{8} = 1.25$ Vertices per Triangle
- **Ergebnis:** wir können Geometriephase mehr als 2x schneller erwarten
- Definition eines Dreieckstreifens bewirkt eine Änderung der Orientierung zwischen benachbarten Dreiecken im Streifen
 - Intern wird die Reihenfolge gegen den Uhrzeigersinn konsistent gehalten, indem die Eckpunkte 0-1-2, 1-3-2, 2-3-4, 3-5-4, ... überquert werden.

Triangle Strips

OpenGL Implementation

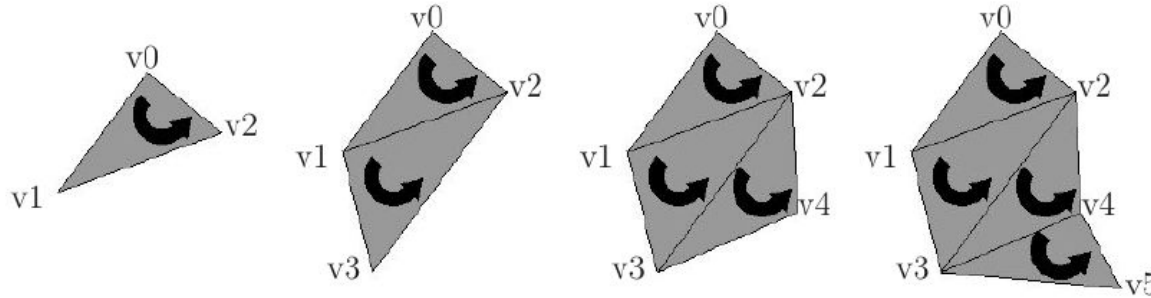


Compatibility Profile

GLSL

```
1  glBegin(GL_TRIANGLE_STRIP)
2      glVertex3fv(v0);
3      glVertex3fv(v1);
4      glVertex3fv(v2);
5      glVertex3fv(v3);
6      glVertex3fv(v4);
7      glVertex3fv(v5);
8  glEnd();
```

Triangle Strips



Core Profile

GLSL

```
1  GLBatch triangleBatch;
2  GLShaderManager SM;
3  float grey[] = {0.5, 0.5, 0.5, 1.0};
4  float verts[][3] = {v0, v1, v2, v3, v4, v5};
5  triangleBatch.Begin(GL_TRIANGLE_STRIP, 6);
6  triangleBatch.CopyVertexData3f(verts);
7  triangleBatch.End();
8  SM.UseStockShader(GLT_SHADER_IDENTITY, grey);
9  triangleBatch.Draw();
```

Triangle Strips

Erstellung

- Manuelle Erstellung - nur bei kleinen Modellen und macht keinen Spaß...
- OpenSceneGraph
- NVIDIA Tri Stip-Library
- Custom Implementationen

Triangle Strips

Erstellung

SGI Stripping Algorithm

1. Wähle ein Start-Dreieck
2. Baue 3 Triangle-Strips (einer pro Edge)
3. Wähle den längsten Strip
4. Beginne bei 1. bis alle Dreiecke in einem Strip sind

Wahl des Start-Dreiecks

- Originale Implementation: Dreieck mit den wenigsten Nachbarn
- Praktisch bringt eine willkürliche Wahl gleich gute Ergebnisse

Triangle Strips

Performance

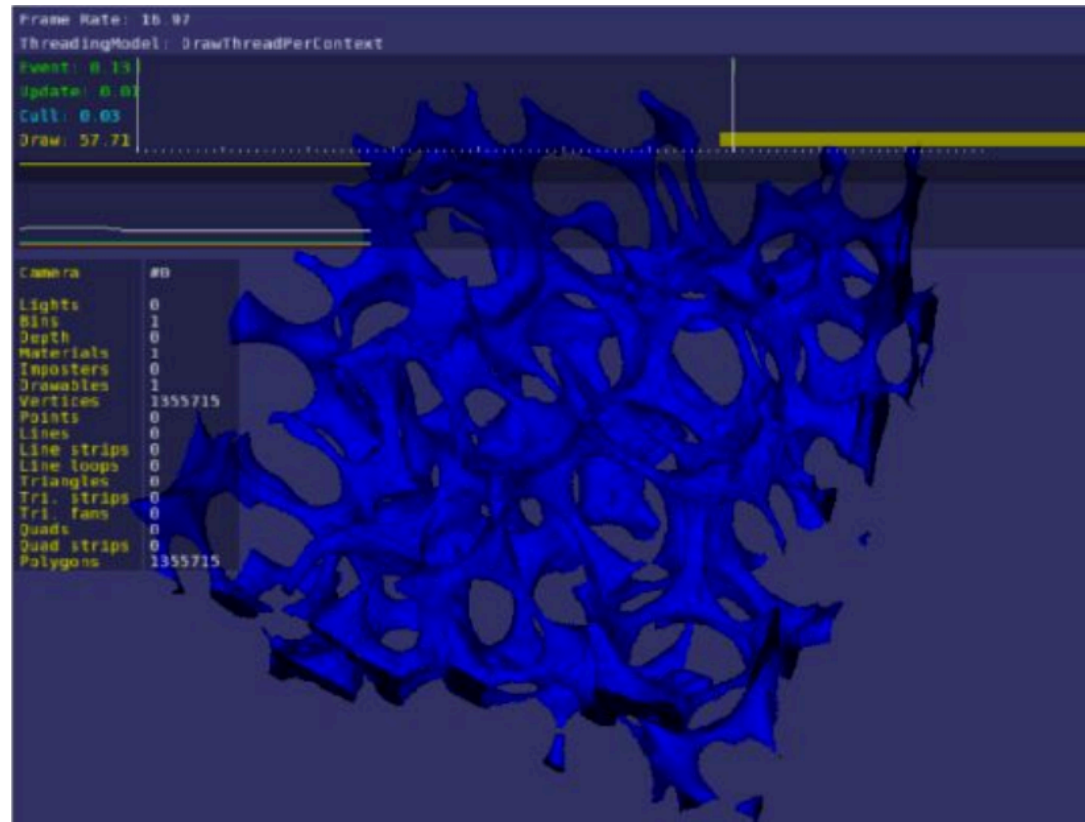


Figure 40: Modell von Siliconcarbidschaum der TU Freiberg

Triangle Strips

Performance

	Originales Modell	Optimiertes Modell
Anzahl der Vertices	ca. 1,36 Millionen	ca. 0,23 Millionen
Anzahl der Primitive	ca. 1,36 Millionen Polygone	ca. 0,44 Millionen Tristrips und ca. 8000 Triangles
Renderdauer	ca. 50 ms	ca. 7 ms
FPS	ca. 19 FPS	60 FPS (maximales Limit)